

ЛАБОРАТОРНАЯ РАБОТА №4

Работа с текстовыми файлами

1. Цель работы

1. Текстовые редакторы.
2. Изучение основных процедур редактирования текста.
3. Сохранение изменений.
4. Обработка текста с помощью фильтров.
5. Использование регулярных выражений.

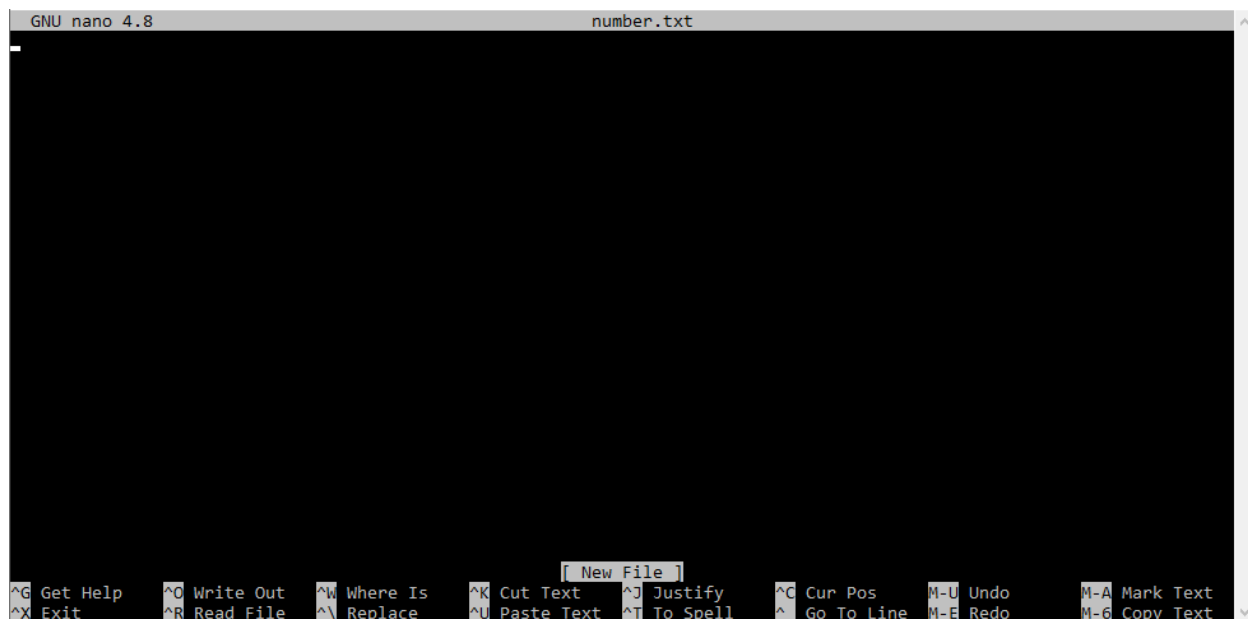
2. Теоретическая часть

Обработка текста выполняется регулярно при управлении системой Linux. Независимо от того, нужно ли изменить файл конфигурации или создать сценарий оболочки, возможность использования интерактивного редактора текстовых файлов в командной строке является важным навыком.

Три популярных текстовых редактора командной строки Linux:

- *emacs*
- *nano*
- *vim*

nano – хороший текстовый редактор, если вы никогда не имели дело с редактором или использовали только GUI редакторы. Чтобы начать использовать текстовый редактор *nano*, введите *nano*, а затем имя файла, который хотите отредактировать или создать. На рисунке показан текстовый редактор *nano* в действии, редактирующий файл с именем number.txt

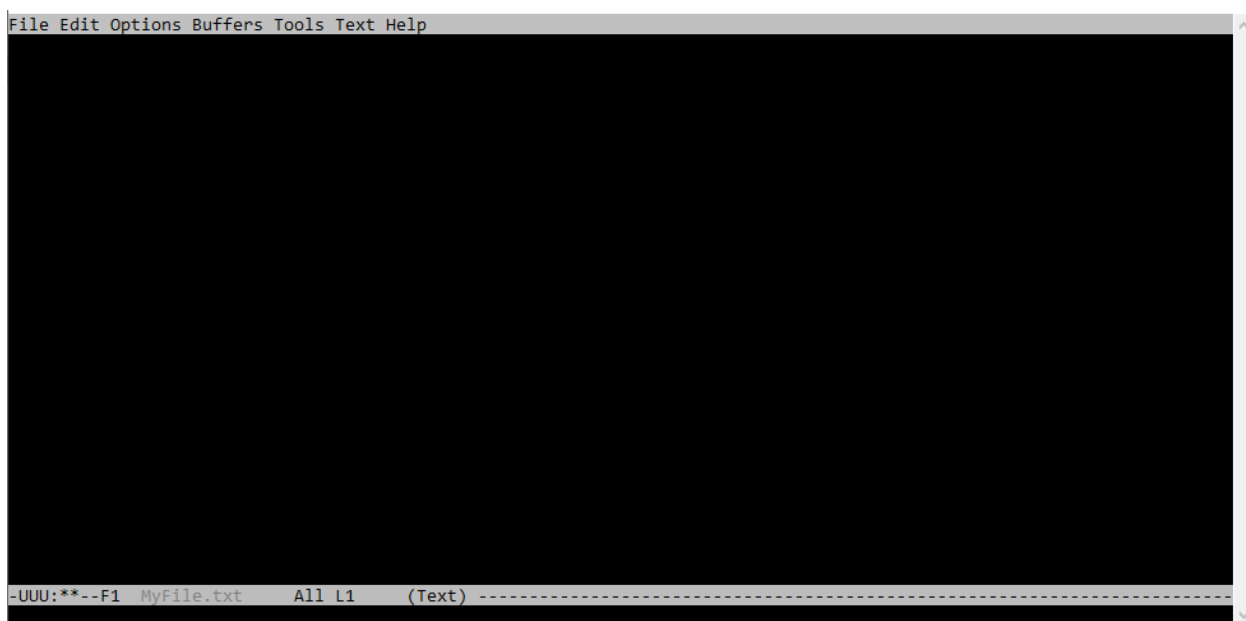


Внизу окна отображаются наиболее распространенные команды и связанные с ними сочетания клавиш. Символ каретки (^) указывает на необходимость использования клавиши Ctrl. Например, чтобы переместиться вниз по странице, нажмите и удерживайте

клавишу Ctrl, а затем нажмите клавишу V. Для получения справки, нажмите комбинацию клавиш Ctrl + G.

Текстовый редактор *nano* замечательно подходит для редактирования простых текстовых файлов. Однако, если нужен более мощный текстовый редактор для создания программ или сценариев оболочки, к популярным вариантам относятся редакторы *emacs* и *vim*.

Чтобы начать использовать текстовый редактор *emacs*, введите *emacs*, а затем имя файла, который хотите отредактировать или создать. На рисунке показан экран текстового редактора *emacs*, редактирующий только что созданный файл с именем MyFile.txt.



Добавление и изменение текста, а также перемещение по этому редактору довольно просты. Однако, чтобы использовать возможности редактора *emacs*, необходимо изучить различные сочетания клавиш. Вот несколько примеров:

- Нажмите Ctrl + X, а затем комбинации клавиш Ctrl + S, чтобы сохранить содержимое буфера редактора в файл.
- Нажмите Ctrl + X, а затем комбинации клавиш Ctrl + C, чтобы выйти из редактора.
- Нажмите комбинацию клавиш Ctrl + H, а затем клавишу T, чтобы перейти к руководству по *emacs*.

Обратите внимание, что в документации редактора *emacs* клавиша Ctrl представлена одной буквой C, и для добавления к ней дополнительной клавиши в документации используется дефис (-) вместо традиционного знака плюс (+).

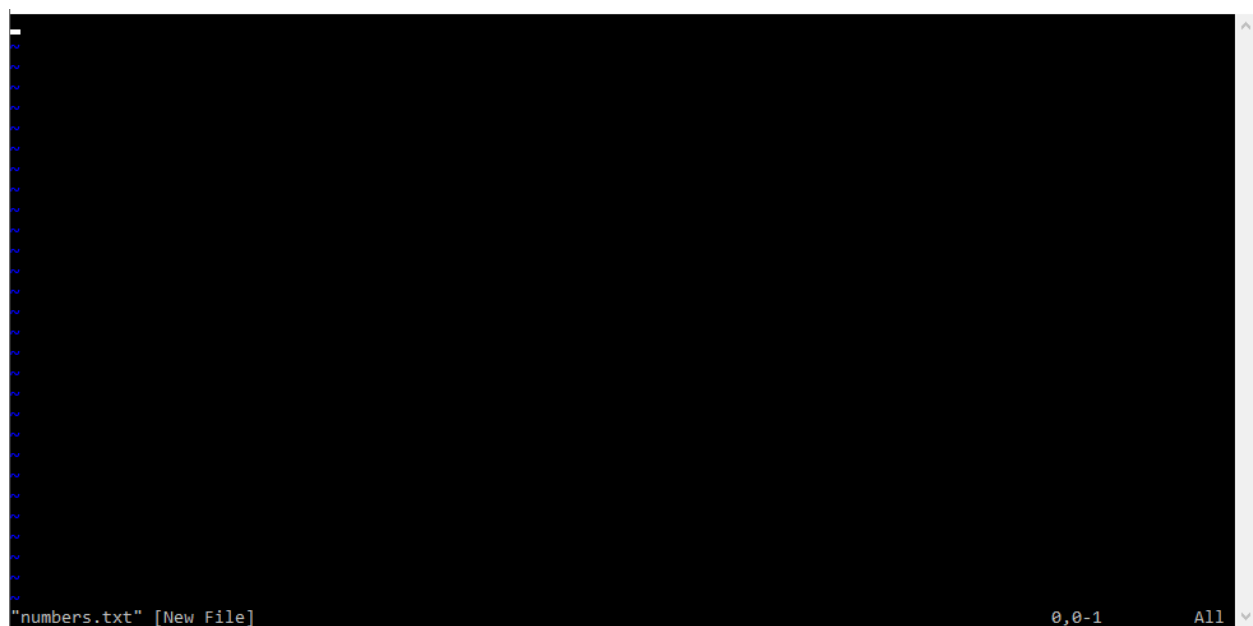
Хотя команды *emacs* немного сложны, преимущества изучения редактора *emacs* включают следующее:

- Команды редактирования, используемые в *emacs*, также можно использовать для быстрого редактирования ваших команд, введенных в командной строке оболочки.
- *emacs* имеет графический аналог со всеми теми же функциями редактирования.
- Можно сосредоточиться на наиболее необходимых функциях редактора и изучить его расширенные возможности позже.

Прежде чем рассмотрим использование редактора *vim*, нужно поговорить о *vim* и *vi*. *vi* был текстовым редактором Unix, и он был переписан как инструмент с открытым исходным кодом. Таким образом, *vim* означает «*vi* улучшенный (*vi improved*)».

Команда *vi* запускает редактор *vim*. В некоторых дистрибутивах только команда *vim* запускает редактор *vim*. Иногда работают обе команды.

Чтобы начать использовать текстовый редактор *vim*, введите *vim* или *vi*, в зависимости от дистрибутива, а затем имя файла, который хотите отредактировать или создать. На рисунке показан экран текстового редактора *vim* в действии.



Если открывать *vim* без имени файла или имя, которое введено, еще не существует, *vim* запускает новую буферную область для редактирования.

В редакторе *vim* есть область сообщений рядом с нижней строкой. Если только что открыли уже созданный файл, он отобразит имя файла вместе с количеством строк и символов, считанных в буферную область. Если создаете новый файл, увидите [Новый файл] в области сообщений.

vim имеет три стандартных режима следующим образом:

- Командный режим. Это режим по умолчанию, который *vim* использует при первом входе в буферную область; иногда его называют нормальным

режимом. Здесь вводите нажатия клавиш для выполнения команд. Например, нажатие клавиши J переместит курсор на одну строку вниз. Командный режим - лучший режим для быстрого перемещения по буферной области.

- Режим вставки. Режим вставки также называется режимом редактирования или ввода. Это режим, в котором вы можете выполнять простое редактирование. Команд или нажатий клавиш в специальном режиме не так много. Переходить в этот режим из командного режима можно, нажав клавишу I. На этом этапе в области сообщений отобразится сообщение «Вставить». Выйти из этого режима можно, нажав клавишу Esc.
- Экс режим. Иногда называют также команды двоеточие, потому что каждая команда вводится здесь после двоеточия (:). Например, чтобы выйти из редактора *vim* и не сохранять никаких изменений, введите :q и нажмите клавишу Enter.

При запуске *vim* в командном режиме полезно понимать некоторые из часто используемых команд для перемещения. Таблица содержит несколько команд перемещения.

Горячие клавиши	Описание
h	Переместите курсор влево на один символ.
l	Переместите курсор вправо на один символ.
j	Переместите курсор на одну строку вниз (следующая строка в тексте).
k	Переместите курсор на одну строку вверх (предыдущая строка в тексте).
w	Переместите курсор вперед на одно слово перед следующим словом.
e	Переместите курсор в конец текущего слова.
b	Переместите курсор назад на одно слово.
^	Переместите курсор в начало строки.
\$	Переместите курсор в конец строки.
gg	Переместите курсор в первую строку файла.
G	Переместите курсор в последнюю строку файла.
[n]G	Переместите курсор на строку файла с номером n .
Ctrl + B	Прокрутите вверх на один полный экран.
Ctrl + F	Прокрутите вниз на один полный экран.
Ctrl + U	Прокрутите вверх половину экрана.
Ctrl + D	Прокрутите вниз половину экрана.
Ctrl + Y	Прокрутите вверх на одну строку.
Ctrl + E	Прокрутите вниз на одну строку.

Есть также несколько команд редактирования, которые помогают ускорить процесс модификации. В следующей таблице перечислены наиболее часто используемые команды редактирования в командном режиме. Обратите особое внимание на регистр каждой буквы, потому что нажатия клавиш в нижнем регистре часто выполняют отличные операции от операций после нажатия клавиш в верхнем регистре.

Горячие клавиши	Описание
a	Вставить текст после курсора.
A	Вставить текст в конец текстовой строки.

dd	Удалить текущую строку.
dw	Удалить текущее слово.
i	Вставить текст перед курсором.
I	Вставить текст перед началом текстовой строки.
o	Открыть новую текстовую строку под курсором и перейти в режим вставки.
O	Открыть новую текстовую строку над курсором и перейти в режим вставки.
p	Вставить скопированный текст после курсора.
P	Вставить скопированный текст перед курсором.
yw	Скопировать текущее слово.
yy	Скопировать текущую строку.

В командном режиме можно продвинуть команды редактирования еще дальше, используя их полный синтаксис, который выглядит следующим образом:

КОМАНДА [КОЛИЧЕСТВО РАЗ] ЭЛЕМЕНТ

Например, если хотите удалить три слова, вы должны нажать клавиши D, 3 и W. Если хотите скопировать текст от курсора до конца текстовой строки, нажать клавиши Y \$, переместиться в то место, куда хотите вставить текст, и нажать клавишу P.

Третий режим *vim*, режим Экс, имеет дополнительные удобные команды. Чтобы войти в режим Экс, должны находиться в командном режиме. Нельзя перейти из режима вставки в режим Экс. Поэтому, если находитесь в режиме вставки, сначала нажмите Esc

В таблице показано несколько команд Экс, которые могут помочь в управлении текстовым файлом. Обратите внимание, что все нажатия клавиш включают двоеточие (:).

Горячие клавиши	Описание
:! [команда]	Выполнить команду оболочки и отобразить результаты, но не выходить из редактора.
:r! [команда]	Выполнить команду оболочки и добавить результаты в буферную область редактора.
:r [файл]	Прочитать содержимое файла и включить его в буферную область редактора.

После того, как внесли необходимые изменения текста в *vim*, пришло время сохранить работу. Можете использовать один из многих методов, показанных в таблице. Введите ZZ в командном режиме, чтобы записать буфер на диск и выйти из редактора *vim*.

Режим	Горячие клавиши	Описание
Экс	:xc	Записать буфер в файл и выйти из редактора.
Экс	:wq	Записать буфер в файл и выйти из редактора.
Экс	:wq!	Записать буфер в файл и выйти из редактора (отменяет защиту).

Экс	:w	Записать буфер в файл и остаться в редакторе.
Экс	:w!	Записать буфер в файл и остаться в редакторе (отменяет защиту).
Экс	:q	Закрыть редактор без записи буфера в файл.
Экс	:q!	Выйти из редактора без записи буфера в файл (отменяет защиту).
Командный	ZZ	Записать буфер в файл и выйти из редактора.

В повседневной работе с Linux полезно знать как минимум два текстовых редактора. Для простых модификаций подходит текстовый редактор *nano*. Для более сложного редактирования предпочтительны редакторы *vim* и *emacs*.

В командной строке Linux часто нужно просматривать файлы или их части. Кроме того, может потребоваться использовать инструменты, позволяющие собирать блоки данных или статистику файлов для устранения неполадок или анализа. Утилиты в этом разделе могут помочь во всех этих действиях.

Полезно собирать короткие текстовые файлы для просмотра на экране и сравнивать их. Основная утилита для просмотра текстовых файлов целиком – это команда конкатенации. Хотя основная цель этого инструмента – объединить текстовые файлы и отобразить их, часто он используется только для отображения одного небольшого текстового файла. Чтобы просмотреть небольшой текстовый файл, используйте команду *cat* с основным синтаксисом:

```
cat [OPTION]... [FILE]...
```

Если хотите отобразить два файла рядом и вас не волнует, насколько небрежным является вывод, можете использовать команду *paste*.

Иногда может понадобиться немного поработать с файлами. Эти ситуации могут включать попытку просмотра графического файла или устранение неполадок текстового файла, который был изменен программой. Утилита *od* может помочь, потому что она позволяет показать содержание файла в восьмеричном (основание 8), шестнадцатеричном (основание 16), десятичном (основание 10), и ASCII форматах. Его основной синтаксис следующий:

```
od [OPTION] ... [FILE] ...
```

По умолчанию *od* отображает текст файла в восьмеричном формате. Первый столбец вывода команды *od* - это порядковый номер для каждой отображаемой строки.

Еще одна удобная команда – *split*. Эта утилита позволяет разделить большой файл на более мелкие части, что удобно, когда хотите быстро создать текстовый файл меньшего размера для тестирования. Базовый синтаксис команды следующий:

```
split [OPTION] ... [INPUT [PREFIX]]
```

Можете разделить файл по размеру, байтам, строкам и так далее. Исходный файл (INPUT) остается без изменений, и в зависимости от выбранных параметров команды создаются дополнительные новые файлы.

Обратите внимание, что для разделения файла по количеству строк необходимо использовать параметр -l (L в нижнем регистре) и указать количество строк текстового файла, которые нужно попытаться поместить в каждый новый файл.

Часто, чтобы понять данные в текстовых файлах, нужно каким-то образом переформатировать данные. Для этого можно использовать несколько простых утилит.

Команда *sort* сортирует данные файла. Имейте в виду, что он не вносит изменений в исходный файл; сортируется только вывод. Основной синтаксис этой команды следующий:

```
sort [OPTION] ... [FILE] ...
```

Если хотите упорядочить содержимое файла, используя стандартный порядок сортировки системы, введите команду сортировки, за которой следует имя файла, который хотите отсортировать. Если файл содержит числа, данные могут быть не в том порядке, который нужен. Чтобы получить правильный числовой порядок, добавьте к команде параметр -n.

Еще одна полезная команда форматирования файлов – утилита *nl*. Эта небольшая команда позволяет пронумеровать строки в текстовом файле. Он позволяет использовать регулярные выражения для обозначения строк, которые нужно пронумеровать. Синтаксис команды является довольно простым:

```
nl [OPTION] ... [FILE] ...
```

Когда работаете из командной строки, просмотр файлов – ежедневное действие. Для короткого текстового файла достаточно использовать команду *cat*. Однако, когда нужно просмотреть большой файл или его часть, доступны другие команды, которые работают лучше, чем *cat*.

Один из способов прочитать большой текстовый файл – использовать пейджер. Утилита пейджера позволяет просматривать по одной текстовой странице за раз и перемещаться по тексту в удобном темпе. Два наиболее часто используемых пейджера - это утилиты *more* и *less*. Синтаксис команды *more* следующий:

```
more [OPTION] FILE [...]
```

Еще один удобный инструмент для отображения частей текстового файла - это утилита **head**. Синтаксис команды следующий:

```
head [OPTION] ... [FILE] ...
```

По умолчанию команда **head** отображает первые 10 строк текстового файла. Если хотите отобразить последние строки файла вместо его первых строк, воспользуйтесь утилитой **tail**. Его общий синтаксис аналогичен синтаксису команды **head**.

Самой простой и наиболее распространенной утилитой для определения счетчиков в текстовом файле является утилита **wc**. Синтаксис команды следующий:

```
wc [OPTION] ... [FILE] ...
```

Когда вводите команду **wc** без параметров и передаете ей имя файла, утилита отображает количество строк, слов и байтов файла в указанном порядке.

Многие команды используют регулярные выражения. Обычное выражение – это шаблон, который определяется для такой утилиты, как **grep**, использующей шаблон для фильтрации текста. Использование регулярных выражений вместе с командами фильтрации текста расширяет навыки работы с командной строкой Linux.

Замечательный инструмент для просеивания текста – команда **grep**. Команда **grep** использует регулярные выражения, которые помогают фильтровать текстовые файлы. Но прежде чем рассмотрим их, ознакомьтесь с таблицей, где указаны наиболее часто используемые параметры утилиты **grep**.

Короткие	Длинные	Описание
-c	--count	Отображение количества записей текстового файла, содержащих совпадение ШАБЛОНА .
-d action	--directories=action	Когда файл является каталогом, если действие установлено на чтение, читать каталог, как если бы это был обычный текстовый файл; если действие настроено на пропуск, игнорировать каталог; и если для действия задано рекурсивное выполнение, действовать так, как если бы использовалась опция -R, -r или --recursive.
-E	--extended-regexp	Обозначить ШАБЛОН как расширенное регулярное выражение.
-i	--ignore-case	Игнорировать регистр в ШАБЛОНЕ, а также в записях любых текстовых файлов.
-R, -r	--recursive	Выполнить поиск содержимого каталога и любого подкаталога в исходном дереве каталогов, а также последовательно выполнить поиск в его содержимом (рекурсивно).
-v	--invert-match	Отображать только те записи текстовых файлов, которые не содержат совпадений с ШАБЛОНОМ .

Синтаксис команды **grep**:

```
grep [OPTION] PATTERN [FILE ...]
```

Команда **grep** возвращает каждую запись (строку) файла, содержащую экземпляр ШАБЛОНА.

Базовые регулярные выражения включают символы, такие как точка, за которой следует звездочка (.*) для обозначения нескольких символов и одна точка (.) для обозначения одного символа. Также можно использовать квадратные скобки для обозначения нескольких символов, таких как [a,e,i,o,u] (необязательно включать запятые) или диапазона символов, например [A-Z].

Чтобы найти записи текстовых файлов, которые начинаются с определенных символов, можно поставить перед ними символ каретки (^). Для поиска записей текстовых файлов, в которых определенные символы находятся в конце записи, добавьте к ним знак доллара (\$). Эти символы называются якорными символами, поскольку они прикрепляют шаблон к началу или концу текстовой строки.

Если нужно отфильтровать все пустые строки в файле (отображать только строки с текстом), используйте команду **grep** с параметром -v, чтобы инвертировать соответствующий шаблон. Затем используйте в командной строке символы привязки ^ и \$, например, **grep -v ^ \$ filename**.

3. Практическая часть

Пример 1: Запустить редактор nano для редактирования текстового файла number.txt:

```
$ nano number.txt
```

Пример 2: Запустить редактор emacs для редактирования текстового файла MyFile.txt:

```
$ emacs MyFile.txt
```

Пример 3: Запустить редактор vim для редактирования текстового файла numbers.txt:

```
$ vim numbers.txt
```

Пример 4: Использовать команду cat для отображения файла:

```
$ cat numbers.txt
```

Пример 5: Использовать команду cat для объединения файлов numbers.txt и random.txt:

```
$ cat numbers.txt random.txt
```

Пример 6: Использовать команду paste для объединения файлов вертикально:

```
$ paste numbers.txt random.txt
```

Пример 7: Использовать команду od для отображения текстового файла в восьмеричном формате:

```
$ od random.txt
```

Пример 8: Использовать команду split для разделения файла по количеству строк:

```
$ split -l 3 numbers.txt numb  
$ ls numb*  
$ cat numbaa
```

Пример 9: Использовать команду sort:

```
$ sort numbers.txt
```

Пример 10: Пронумеровать строки файла:

```
$ nl numbers.txt
```

Пример 11: Вывести первые 2 строки файла numbers.txt:

```
$ head -2 numbers.txt"
```

Пример 12: Вывести статистическую информацию файла numbers.txt:

```
$ wc numbers.txt
```

Пример 13: Поиск текста «123» во всех файлах в текущем каталоге:

```
$ grep 123 *
```

Пример 14: Поиск текст, начинающегося с «1» и заканчивающегося на «3» во всех файлах текущего каталога:

```
$ grep ^1.*3$ *
```

4. Задание

1. В редакторе nano создать текстовый файл ВашаФамилия.txt, записать в него 10 случайных чисел. Сохранить файл и выйти из редактора.
2. В редакторе emacs создать текстовый файл ВашеИмя.txt, записать в него 10 случайных чисел. Сохранить файл, выйти из редактора.
3. В редакторе vim создать текстовый файл ВашиИнициалы.txt, записать в него содержимое файлов ВашаФамилия.txt и ВашеИмя.txt. Сохранить файл, выйти из редактора.
4. Вывести на экран содержимое всех трех файлов.
5. Объединить файлы ВашаФамилия.txt и ВашеИмя.txt вертикально.
6. Вывести файл ВашиИнициалы.txt в шестнадцатеричном формате.
7. Разделить файл ВашиИнициалы.txt по 2 строки, список полученных файлов и их содержимое показать на экране.
8. Отсортировать содержимое файла ВашиИнициалы.txt как числа по убыванию.
9. Пронумеровать все (в том числе пустые) строки файла ВашиИнициалы.txt.
10. Вывести первые 2 и последние 3 строки файла ВашиИнициалы.txt.
11. Вывести статистическую информацию файла ВашиИнициалы.txt.
12. Найти текст, начинающийся с «1» и заканчивающийся на «3» во всех файлах в текущем каталоге.